
Uncertainty-Aware KV Cache Compression for Training-Free Efficient Language Model Inference

范文茜
524531910002

裘嘉敏
524030910068

陆紫萌
524531910001

Abstract

The KV cache compression during transformer inference can be unreliable when token importance differs across KV heads. We propose an uncertainty-aware wrapper for score-based KVPress methods that augments each token’s average importance with a head-wise variance term, allowing the compressor to preserve tokens whose relevance is uncertain across heads. The method is training-free, simple to implement, and compatible with multiple existing presses. Experiments on Pythia-70m using WikiText-2 and PG-19 show that the uncertainty term substantially improves KNorm compression, reducing WikiText-2 perplexity from 50.52 to 42.46 at a 0.5 compression ratio, while providing limited or negative gains for stronger base presses such as SnapKV, CUR, and LagKV. These results suggest that head-wise disagreement is a useful correction signal for simple static scoring methods, but its benefit depends on the base compression mechanism.

1 Introduction

Key-Value (KV) cache compression is a practical training-free approach for transformer optimization. However, a token with moderate average importance but large head-wise disagreement may still be risky to prune, because some heads may rely on it strongly. Motivated by the observation that token importance is not always agreed upon across KV heads (Ge et al., 2024), we propose an uncertainty-aware KV cache compression wrapper for score-based presses in KVPress. It reuses a base scorer and augments its token ranking with a head-disagreement term. Given per-head token scores, we compute the mean importance across heads and the variance across heads, then rank tokens by

$$s_i = \text{Norm}(\mathbb{E}_h[s_{h,i}]) + \lambda \text{Norm}(\text{Var}_h[s_{h,i}]),$$

where λ controls the degree of risk aversion. This design is simple, training-free, and compatible with multiple existing score-based KV compression methods. Intuitively, the method keeps tokens that are either consistently important across heads or uncertain because different heads assign them very different scores.

2 Method

We propose an uncertainty-aware KV cache compression method that wraps an existing score-based KVPress and accounts for disagreement between KV heads when estimating token importance. Instead of pruning only by the base importance score, we augment the score with an uncertainty term estimated from inter-head variance.

Let the base press produce token scores

$$S \in \mathbb{R}^{B \times H \times L},$$

where B is the batch size, H is the number of KV heads, and L is the sequence length. These scores can come from different base methods. For each token position t , we compute the mean score across

KV heads:

$$\mu_t = \text{mean}_h(S_{h,t}),$$

and define the uncertainty term as the head-wise variance of the base scores:

$$u_t = \text{Var}_h(S_{h,t}).$$

The adjusted token score is then

$$a_t = \text{norm}(\mu_t) + \lambda \cdot \text{norm}(u_t),$$

where λ is the uncertainty weight. A larger λ makes the compressor more conservative toward tokens whose importance is uncertain across heads. In the implementation, both the mean score and the uncertainty term are min-max normalized along the sequence dimension, with a small ϵ for numerical stability. This makes the combination less sensitive to the raw score scale of different base presses.

After computing the adjusted score, the same token-level score is expanded back to all KV heads. The standard `ScorerPress` compression mechanism is then used: for a target compression ratio, the method keeps the top-scoring KV positions and removes the lowest-scoring ones during refill.

3 Experiments

We evaluate whether the proposed uncertainty-aware score adjustment improves KV cache compression under different base presses. Our experiments focus on three questions: (i) whether uncertainty-aware adjustment reduces perplexity under the same compression ratio; (ii) whether the improvement is consistent across different scoring mechanisms; and (iii) whether the additional scoring computation affects inference efficiency.

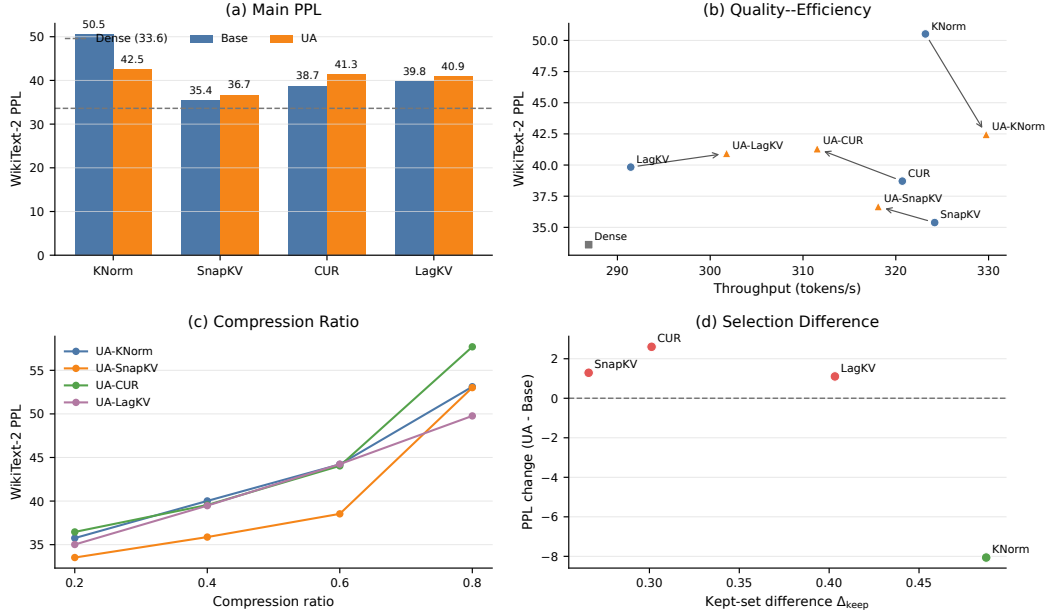


Figure 1: Experimental summary of uncertainty-aware KV cache compression. (a) Main perplexity comparison between base presses and their uncertainty-aware variants. (b) Quality-efficiency trade-off measured by perplexity and throughput. (c) Perplexity under different compression ratios. (d) Retained-token set difference versus perplexity change, where negative PPL change indicates improvement.

3.1 Experimental Setup

Model. All experiments are conducted on EleutherAI/pythia-70m. We use the same model checkpoint for all methods to ensure a controlled comparison. **Datasets.** We evaluate language

modeling quality on WikiText-2 and PG-19. WikiText-2 is used as the primary benchmark for perplexity evaluation, while PG-19 is used to test long-context behavior. **Compared methods.** The original kvpress methods we apply include KNorm, CURPress, SnapKVPress, and LagKVPress. We compare these methods with their uncertainty-aware variants. We also include a dense baseline without KV cache compression. All compression methods are implemented under the kvpress framework. **Compute.** All experiments are conducted on a single NVIDIA GeForce RTX 4060 GPU using CUDA. **Default evaluation parameters.** Unless otherwise specified, all experiments use the same evaluation configuration shown in Table 1.

Table 1: Default evaluation parameters used in our experiments.

Parameter	Default Value	Description
dataset	wikitext-2	Dataset dataset used for evaluation
split	test	Dataset split used for evaluation
num-samples	10	Number of evaluated samples
warmup-runs	2	Number of warmup runs
context-tokens	1024	Number of context tokens used for compression
target-tokens	256	Number of target tokens used for PPL evaluation
max-new-tokens	64	Maximum number of generated tokens for speed evaluation
compression-ratio	0.5	Fraction of KV cache entries removed
uncertainty-weight	1.0	Weight λ for the uncertainty term

3.2 Main Results

Table 2 reports the main results under the default compression ratio $r = 0.5$. Compared with vanilla KNorm, UA-KNorm reduces WikiText-2 perplexity from 50.52 to 42.46, corresponding to a relative improvement of 15.96%. However, for SnapKV, CUR, and LagKV, the uncertainty-aware variants do not improve perplexity under the default setting.

In terms of efficiency, the uncertainty-aware wrapper does not change the target cache size, so the decoding-time cost remains close to the corresponding base press. This supports that the additional mean-variance computation mainly affects the scoring stage rather than the compressed decoding process.

Table 2: Main results on WikiText-2 and PG-19 under compression ratio $r = 0.5$. Lower PPL, TTFT, and TPOT are better; higher throughput is better. TTFT and TPOT are reported in milliseconds.

Method	Ratio	WikiText-2 PPL ↓	PG-19 PPL ↓	TTFT (ms) ↓	TPOT (ms) ↓	Throughput ↑
Dense	0.0	33.61	32.10	12.85	3.31	286.90
KNorm	0.5	50.52	39.45	13.64	2.90	323.19
UA-KNorm	0.5	42.46	33.83	14.69	2.82	329.74
SnapKV	0.5	35.38	34.10	16.25	2.85	324.19
UA-SnapKV	0.5	36.67	36.28	17.73	2.88	318.11
CUR	0.5	38.71	37.57	14.96	2.90	320.70
UA-CUR	0.5	41.31	40.71	17.15	2.96	311.53
LagKV	0.5	39.83	32.56	17.89	3.17	291.43
UA-LagKV	0.5	40.94	33.09	17.43	3.06	301.76

3.3 Effect of Compression Ratio

We further evaluate the uncertainty-aware variants under different compression ratios. Table 3 shows the results.

As expected, throughput generally improves as the compression ratio increases, although the trend is not perfectly monotonic for every method. This is because a larger ratio removes more KV cache entries, reducing attention computation and memory access during generation. However, the same increase in ratio also worsens perplexity, so the compression ratio should be chosen as a trade-off between quality and efficiency.

Table 3: Perplexity and throughput on WikiText-2 under different compression ratios. Lower PPL is better; higher throughput is better.

Method	$r = 0.2$		$r = 0.4$		$r = 0.6$		$r = 0.8$	
	PPL ↓	Throughput ↑	PPL ↓	Throughput ↑	PPL ↓	Throughput ↑	PPL ↓	Throughput ↑
UA-KNorm	35.75	295.11	40.02	311.80	44.21	306.06	53.12	302.56
UA-SnapKV	33.52	285.96	35.87	299.56	38.53	306.58	53.01	310.11
UA-CUR	36.46	291.04	39.55	299.25	44.04	317.12	57.70	318.10
UA-LagKV	35.02	295.04	39.47	307.41	44.24	308.62	49.77	314.59

3.4 Ablation Study

Effect of uncertainty weight. Table 4 studies the effect of the uncertainty weight λ . For UA-KNorm, adding the uncertainty term consistently improves over $\lambda = 0$. The PPL decreases from 43.13 at $\lambda = 0$ to 40.99 at $\lambda = 0.5$ and further to 40.65 at $\lambda = 2.0$. This confirms that head-wise variance is a useful correction signal for KNorm. For UA-SnapKV, UA-CUR, and UA-LagKV, changing λ has little effect or slightly worsens PPL, suggesting that these methods already capture token importance through their original scores and that head-wise variance provides little, or even negative, additional signal.

Table 4: Ablation on uncertainty weight λ on WikiText-2 under compression ratio $r = 0.5$. Lower PPL is better.

Method	$\lambda = 0$	$\lambda = 0.25$	$\lambda = 0.5$	$\lambda = 1.0$	$\lambda = 2.0$
UA-KNorm	43.13	42.38	40.99	42.46	40.65
UA-SnapKV	37.03	37.20	36.94	36.67	36.65
UA-CUR	39.63	40.83	41.96	41.31	41.13
UA-LagKV	40.97	40.73	40.73	40.94	41.50

Effect of normalization. Table 5 evaluates the normalization choices using UA-KNorm. These results show that normalization is not always beneficial for KNorm. The raw score magnitude itself may already contain useful information about token importance. Over-normalizing the score distribution may weaken this information and make the final ranking less faithful to the original key-space geometry. Therefore, although normalization makes the method more comparable across different base presses, it may not be optimal for every individual press.

Table 5: Ablation on mean-score and variance normalization for UA-KNorm on WikiText-2. Lower PPL is better.

Variant	Normalize Mean	Normalize Variance	PPL ↓
Full UA	✓	✓	42.46
w/o mean normalization	×	✓	42.07
w/o variance normalization	✓	×	40.91
Raw combination	×	×	41.29

3.5 Discussion

The results show that uncertainty-aware adjustment is not uniformly effective across base presses. The uncertainty term is most effective when the original score lacks dynamic attention information, as in KNorm. For stronger base methods whose scores already encode token salience through attention patterns, matrix leverage, or lag-relative information, the head-wise variance term may introduce additional noise rather than useful correction. Another possible reason is that SnapKV, CUR, and LagKV originally produce per-head token scores, so different attention heads can retain different tokens according to their own salience estimates. After applying the uncertainty-aware wrapper, these per-head scores are reduced to a shared mean-variance token score, eliminating potentially useful differences between heads.

References

Suyu Ge, Yunan Zhang, Liyuan Liu, Minjia Zhang, Jiawei Han, and Jianfeng Gao. Model tells you what to discard: Adaptive KV cache compression for LLMs. In B. Kim, Y. Yue, S. Chaudhuri, K. Fragkiadaki, M. Khan, and Y. Sun, editors, *International Conference on Learning Representations*, volume 2024, pp. 22975–22988, 2024. URL https://proceedings.iclr.cc/paper_files/paper/2024/file/639a9a172c044fbb64175b5fad42e9a5-Paper-Conference.pdf.

A Contribution Statement

The work was completed collaboratively by all group members. Wenqian Fan was responsible for the implementation of the uncertainty-aware KV cache compression method and writing the method description. Jiamin Qiu was responsible for running experiments, collecting results, and analyzing the data. Zimeng Lu was responsible for writing and revising the report, including the introduction, method description, and discussion.

B Group Member Repositories

- Wenqian Fan: <https://github.com/sissiwqwt/transformer-optimizer.git>
- Jiamin Qiu: <https://github.com/JaneQiu2006/nlp-project-KVPress.git>
- Zimeng Lu: https://github.com/mandyl-cyber/homework_for_nlp.git